Title: Parallel Graph-Based Approximate Nearest-Neighbor Search on GPUs

Team: Mu-Wei Hsieh, Ryder (ShengHua) Chou
Project webpage: https://github.com/morweee/15618-parallel-ann-search

Summary

We are going to implement graph-based nearest-neighbor search on both GPU and multi-core CPU platforms, and analyze how different ways of mapping queries onto GPU threads affect performance.

Background

In a given dataset, nearest-neighbor search finds the points that are closest or most similar to a specific point under a chosen distance metric. It takes an input vector, called a query, and finds the stored vectors closest to it. It is used in applications such as image similarity, recommendation, and information retrieval. Approximate search avoids checking every stored vector by allowing some loss in retrieval accuracy.

We will search a graph whose vertices represent vectors and whose edges connect nearby vectors. Starting from an entry vertex, the search repeatedly expands candidates, evaluates their neighbors, and keeps the best candidates in a list of size L. As in BFS, a visited set is maintained to avoid repeated work. The search returns ten neighbors, and we measure quality using recall@10.

The focus is query execution. We will build the index offline with hnswlib and export its bottom layer into adjacency arrays, which are easy for the GPU to read. Our CPU and GPU implementations will use the same graph and search strategy. Parallelism is available across independent queries, across the neighbors of a vertex, and across the dimensions of each distance calculation.

The Challenge

Unlike BFS, which can expand a whole level of the graph at once, this search always expands the single best candidate next, so each query is a chain of dependent steps: the next vertex to expand depends on the distances computed in the current step. In addition, neighbor vectors are scattered in memory, and queries take different numbers of steps, leading to divergence and load imbalance.

The main design tradeoff is how much parallel work to assign to each query. One thread per query allows many independent searches, but gives no cooperation within a query and can produce scattered memory accesses. One warp per query lets 32 threads share distance calculations, but introduces coordination and uses more threads per search. Small batches may provide too little parallelism for either design to use the GPU effectively.

Candidate lists and visited sets become another concern, because fast on-chip storage is limited, while storing them in global memory adds access costs. We will explore how these choices affect latency and throughput to understand work decomposition, locality, occupancy, and load balancing.

Resources

We will run our code on the GHC machines used in previous assignments (8-core Intel i7-9700, an RTX 2080 with 8 GB of memory, and CUDA 11.7). I (Morris) have a personal RTX 4080 laptop that serves as a backup (its results will be reported separately). We will use Nsight tools for profiling.

We will use the HNSW paper as a reference for graph construction and search, and its C++ implementation, hnswlib, to pre-build the index. We will write the search code from scratch; hnswlib is used only to build the graph and as a reference. The CAGRA paper provides related work on GPU graph search.

Data: SIFT1M (1M vectors, 128 dimensions, 10K queries) and GIST1M (1M vectors, 960 dimensions, 1K queries), with exact nearest neighbors provided by ANN-Benchmarks.

- HNSW paper: https://arxiv.org/abs/1603.09320
- hnswlib: https://github.com/nmslib/hnswlib
- CAGRA paper: https://arxiv.org/abs/2308.15136
- ANN-Benchmarks: https://github.com/erikbern/ann-benchmarks

Goals and Deliverables

Plan to achieve

1. **Validate the graph and search.** Test whether the exported graph reaches recall@10 of at least 0.95 on SIFT1M at practical search effort, with an early check on GIST1M. Compare against full hnswlib search to assess the cost of removing the upper layers.
2. **Implement four versions.** Implement sequential C++, multicore C++ with OpenMP, CUDA with one thread per query, and CUDA with one warp per query, and compare their performance.
3. **Evaluation experiments:**

Experiment	Comparison	Analysis
Recall vs. throughput	Sweep several candidate-list sizes on SIFT1M	Speed vs. retrieval quality
Batch size	Test 1, 32, 256, 1,000, and all evaluation queries	The point at which GPU execution becomes worthwhile
Dataset	Compare SIFT1M and GIST1M at a common batch size	Whether the results change on a different dataset with longer vectors
Profiling	Examine both GPU kernels at batch 32 and full batch	What limits performance

We will use a small set of queries to choose search settings that reach at least 95% recall (recall@10 ≥ 0.95), then evaluate on separate queries. These settings stay fixed when testing different batch sizes, and we use the same batch size when comparing datasets. We will also test a few CUDA block sizes before the main experiments.

Measurement: we will report throughput, single-query latency, and batch completion time, separating GPU kernel time from end-to-end time (including query and result transfers). The graph is loaded into GPU memory once and reused for all queries. Profiling will examine memory behavior, occupancy, and divergence, along with per-query workspace and distance-evaluation counts.

Our goal is to identify where cooperative GPU search improves performance and explain where it does not. We expect warp cooperation to help some workloads, but we do not assume a fixed speedup yet. Deliverables include code, reproducible experiments, and plots in the final report.

Hope to achieve

- Use multiple warps per query to improve small-batch latency.
- Use a small visited set in shared memory (a bounded hash table) instead of global memory. This changes the algorithm slightly, since a vertex may be visited twice, so we will report its effect on recall as well as on speed.

If the work goes slowly, we will complete both CPU versions and one GPU design on SIFT1M only, with recall, batch-size, and profiling results. Explaining the measured bottlenecks takes priority over adding GIST1M. We also recognize a graph risk that we need to verify early in the project: searching only the bottom layer may take too many steps to find accurate results. We plan to test this before starting the GPU work. If needed, we will try a better starting vertex or more edges per vertex. As a fallback, we can use the CPU to search the upper layers for a starting point, then let the GPU search the bottom layer. In that case, both stages and the data transfers will be included in the total runtime.

Platform Choice

C++ lets the CPU and GPU versions share the same data layout and loader. CUDA gives direct control over threads, warps, shared memory, and synchronization, which is what we want to compare.

The workload has many independent queries and many distance calculations over large vectors, which suits a GPU's high memory bandwidth, but the dependent traversal and irregular reads make the speedup uncertain. The CPU on the same machine provides a useful baseline, particularly for small batches. The project targets a single GPU with the index resident in GPU memory; multi-GPU and out-of-core search are out of scope.

Schedule

Week	Planned work
1 (Nov 9)	Submit the proposal on Nov 9. Prepare datasets, export the graph, and implement the serial reference. Complete the Nov 12 graph-feasibility checkpoint.

Week	Planned work
2 (Nov 16)	Implement the OpenMP and thread-per-query CUDA versions. Check correctness, collect initial timings, and begin the warp version. Prepare the milestone report.
3 (Nov 23)	Submit the milestone on Nov 23. Keep Thanksgiving week lighter: complete and check the warp version, run initial profiling, and finalize experiment scripts.
4 (Nov 30)	Run the main experiments, investigate bottlenecks, and prepare plots. Attempt stretch goals only after core results are complete.
5 (Dec 7)	Finish measurements, submit the report on Dec 10, and present the poster on Dec 11.

We will divide implementation and evaluation work between both partners and update the schedule weekly. The poster will show the thread mappings, performance curves, and evidence explaining the main bottlenecks.